

Does the Source of Carrier Image Affect Steganographic Detectability?

Midway Project Proposal

Project 2.2, Department of Advanced Computing Sciences

Abdul Moiz Akbar Daria Gjonbalaj Nico Müller-Späth
Jimena Narvaez del Cid David Wicker Nikolas Zouros

Maastricht University · February 2026

1. Motivation and Problem Statement

Image steganography is an important topic in online security and privacy, with both beneficial and adversarial uses [1, 2, 14]. In practice, detectability depends not only on the embedding method but also on the underlying statistics of the carrier image [3, 4]. Steganalysis works by detecting subtle distributional perturbations in pixel values, residuals, and frequency coefficients after embedding. This is exactly why carrier selection matters: two carriers with identical payload and identical embedding can produce different detectability outcomes.

Most existing steganalysis pipelines were developed and benchmarked on real camera photographs, where sensor noise and compression traces are relatively well characterized. This assumption is increasingly fragile. Photorealistic synthetic images from modern text-conditional diffusion generators (e.g., Stable Diffusion variants) are now widespread [6], and they are produced by different mechanisms than camera pipelines. Prior synthetic-image forensics shows that generated images can contain distinct statistical traces [16, 17], which may directly influence steganographic detectability.

The core question is therefore: **do steganalysis detectors validated on photographs remain equally effective on ML-generated carriers under identical embedding settings?** This matters for three reasons:

- **Security:** if ML-generated carriers are harder to detect, an attacker may evade existing detectors by changing carrier source only.
- **Scientific:** the interaction between generative-model distributions and embedding perturbations remains underexplored.
- **Practical:** as AI-generated images become common in communication channels, the steganographic attack surface expands.

The problem statement for this project is to quantify whether *carrier source category* (real vs. ML-generated, and model-to-model within ML) causes measurable detection differences when embedding, payload, and detector settings are fixed. If source effects are large, detector calibration and risk assumptions built on photo-only datasets may not transfer to mixed real/synthetic traffic.

The closest precedent is De et al. [10], who demonstrate AI-generated images for steganographic secret sharing, but without a controlled real-vs-ML carrier comparison using standardized LSB/DCT pipelines and classical ste-

ganalysis endpoints. In other words, feasibility of secret sharing on generated images has been shown, but comparative detectability under matched conditions is still unclear.

This study addresses that gap with a controlled 3×2×3×2 design over 1,500 carrier images (500 real, 500 ML-A, 500 ML-B), two embedding methods, three payload levels, and three detector families (cross-domain baseline, LSB-specific, and DCT-specific). By keeping embedding, payload, and detector settings fixed while varying source category, we isolate whether source and generation model are meaningful drivers of detection-performance differences. Results will guide benchmarking and steganalysis policy in mixed real/synthetic environments.

2. Research Questions

RQ1 (Carrier Source: Real vs. ML)

Does carrier source (real photographs vs. ML-generated images) significantly affect steganographic detectability under matched embedding settings?

RQ2 (Generation-Model Effect Within ML)

Within ML-generated carriers, does the choice of text-conditional generation model (ML-A vs. ML-B) significantly affect detectability under prompt-matched conditions?

RQ3 (Payload Interaction)

Does payload size alter source-related detectability gaps (real vs. ML and ML-A vs. ML-B)?

RQ4 (Embedding Method)

Do different embedding methods (spatial LSB vs. frequency-domain DCT) interact differently with carrier source in terms of detectability?

RQ5 (Encryption Interaction)

Does payload encryption prior to embedding change detectability, and does this effect depend on carrier source?

Verification details for each RQ are specified in Section 4.

3. Chosen Approaches

3.1 Datasets

Real images (500 total). To enforce caption quality, we use only datasets that already provide detailed human-written captions per image: **COCO** [19] (300 images) and **Flickr30k** [20] (200 images). All selected images are normalized to 512×512, RGB, 8-bit PNG.

Caption curation. Each selected real image is assigned one complete, relevant caption (subject + key scene context), chosen from the dataset-provided human captions only. No generated captions are used. Images with ambiguous, underspecified, or low-information captions are excluded.

ML-generated images (1,000 total). From the curated 500-image real set, we generate two prompt-matched sets: **ML-A** (500) with Stable Diffusion XL 1.0 [7] and **ML-B** (500) with PixArt- α [8], both via **diffusers**. This intentionally compares two different generator families (latent U-Net diffusion vs. diffusion-transformer). Each real-image caption is used once per model, yielding paired prompts.

Generation-stage quality control. Generated candidates are filtered before embedding using a BRISQUE gate plus artifact rejection, then normalized to a single 512×512 RGB PNG format used by all embedding and detection branches.

3.2 Embedding Methods

We implement two canonical steganographic methods spanning the two principal domains:

LSB substitution (spatial domain) replaces the k least significant bits of each pixel channel value with pseudorandom message bits, using a PRNG-keyed pixel selection mask. We test $k = 1$ (Low, Medium payload) and $k = 2$ (High payload).

DCT-based embedding (frequency domain) partitions each image channel into non-overlapping 8×8 blocks, computes the 2D DCT, and embeds bits into mid-frequency coefficients (zigzag positions 10–54) via **Quantization Index Modulation (QIM)** [15]:

$$C'_i = \Delta \cdot \text{round}\left(\frac{C_i}{\Delta}\right) \pm \frac{\Delta}{4},$$

where the sign encodes the message bit. DCT embedding is applied directly to the same PNG-format carriers (no format conversion), so comparisons with LSB are not confounded by codec differences. It is chosen alongside LSB to test whether frequency-domain methods are more sensitive to carrier source (RQ4), since DCT coefficients reflect the generative model’s learned spectral distribution directly.

3.3 Encryption

To test RQ5, payloads are processed under two conditions before embedding: plain and AES-256-CBC encrypted. The same encryption setting is applied uniformly across both LSB and DCT embedding branches, so any detectability difference can be attributed to encryption status rather than method-specific preprocessing.

3.4 Image-Quality Tests (Quality Control)

Image quality is treated as a validity control at generation, post-embedding, and post-encryption stages. For each stego/cover pair we compute PSNR, SSIM, and FSIM; BRISQUE is used as a no-reference generation gate. Metrics are reported by source group, embedding method, and payload level to rule out trivial degradation effects.

3.5 Steganalysis Detectors

Our detector selection is deliberately scoped to *classical signal processing and statistics*, consistent with this project’s cryptography/steganography focus:

Table 1: Steganalysis detector comparison.

Detector	Type	Training	Domain
SRM+EC [4, 5]	Classical ML	Labeled	LSB + DCT
RS Analysis [11]	Statistical	None	LSB only
χ^2 attack [12]	Statistical	None	LSB (sensitivity)
Block-DCT shift test [13]	Statistical	None	DCT only

SRM + Ensemble Classifier (EC) [4, 5] is the primary classical baseline used in all conditions. We extract full SRM residual co-occurrence features and train an ensemble of FLD base learners on bootstrap-resampled feature subspaces (the canonical SRM+EC setup), using source-stratified train/test splits to avoid leakage.

RS + χ^2 analytical checks [11, 12] are applied only to LSB experiments as fast, training-free detectors.

Block-DCT shift test [13] is applied only to DCT experiments as a training-free counterpart to RS. It measures embedding-induced shifts in block-DCT coefficient distributions and quantization residues relative to matched cover statistics.

Optional extension. If compute permits, we add one deep detector (SRNet [22]) as a secondary benchmark and report it separately from the primary classical results.

4. Experiments

Exp. 1 (RQ1: Carrier Source, real vs. pooled ML). Apply all applicable detectors to all 1,500 carriers across payload levels, methods, and encryption conditions (LSB: SRM+EC, RS, plus χ^2 sensitivity; DCT: SRM+EC, block-DCT shift test). Compute AUC per source group and compare real vs. pooled ML (ML-A + ML-B) performance under matched settings using Wilcoxon signed-rank and Cohen’s d .

Exp. 2 (RQ2: Generation-model effect within ML). Using prompt-matched ML pairs, compare AUC between ML-A and ML-B under identical embedding/encryption/payload settings. Test whether generation model induces a significant detectability difference, with detector-specific reporting and effect sizes.

Exp. 3 (RQ3: Payload interaction). Plot AUC and source-contrast gaps across Low/Medium/High payload. Test source $\times$ payload interaction and evaluate whether payload modulates both real-vs-ML and ML-A-vs-ML-B detectability contrasts.

Exp. 4 (RQ4: Embedding-method interaction). Run ANOVA on AUC with factors source category and embedding method (LSB/DCT). A significant source $\times$ method interaction indicates method-dependent detectability differences across source groups.

Exp. 5 (RQ5: Encryption interaction). For each source group and embedding method, compare AUC between plain and AES-256-CBC payload conditions. Test both the encryption main effect and source $\times$ encryption interaction.

5. Prototype

Vertical prototype (algorithm depth): We validate the core components separately before scaling: LSB embedding + RS/ χ^2 , DCT-QIM + block-DCT shift test, and SRM+EC on a small test subset.

Horizontal prototype (integration breadth): We then run an integrated pipeline on a mixed 60-image subset (20 real + 20 ML-A + 20 ML-B) at medium payload to validate interfaces and outputs before executing the full 1,500-image study.

6. Related Work

6.1 Classical Embedding and Detection Foundations

Image steganography has predominantly focused on embedding secret data into photographic carriers using spatial- and frequency-domain techniques [1–3]. Canonical examples include LSB substitution, which perturbs pixel intensities directly, and DCT-based Quantization Index Modulation (QIM), which embeds information in transform coefficients [15]. We adopt these established post-hoc embedding strategies rather than introducing a new embedding algorithm, to preserve comparability with prior steganalysis literature and isolate the effect of carrier origin.

On the detection side, RS analysis and the χ^2 attack provide analytical baselines for LSB-like embedding [11, 12]. Rich-model approaches remain strong classical baselines: Fridrich and Kodovský [4] introduced SRM features that capture high-dimensional residual co-occurrences linked to embedding artifacts.

6.2 Generative and Coverless Steganography

Recent generative steganography methods use GAN- or diffusion-based models to generate stego-images directly from secret messages [24, 25]. Related coverless approaches encode information by selecting or generating content rather than modifying a fixed image [26]. Our setup differs in a key way: ML-generated images are treated as *passive carriers*, and standardized post-hoc embedding (LSB and DCT) is applied afterward. This separation between generation and embedding lets us test carrier-origin effects under matched embedding conditions.

6.3 AI-Generated Carriers and Closest Prior Work

The closest related study is De et al. [10], which demonstrates AI-generated photorealistic images for steganographic secret sharing. However, it does not perform a controlled real-versus-ML comparison under identical embedding pipelines, does not compare multiple generation models under prompt-matched conditions, and does not evaluate detectability using classical steganalysis endpoints such as ROC-AUC. Our factorial design (source category $\times$ embedding method $\times$ payload size $\times$ encryption) makes steganalysis detectability the primary outcome.

6.4 Cross-Domain and Synthetic-Image Forensics

Cross-domain steganalysis has typically examined distribution shifts between camera sources [4]. The real-versus-synthetic shift is broader, because the full image-generation process changes when moving from camera capture to GAN or diffusion synthesis. Synthetic-image forensics reports distinct statistical traces in generated images [16, 17], supporting our hypothesis that carrier origin can affect steganographic detectability.

6.5 Classical vs. Deep Steganalysis

Recent deep-learning steganalysis methods can achieve strong benchmark accuracy [21, 22], but often with higher computational requirements and reduced interpretability. We therefore prioritize classical detectors (SRM+EC, block-DCT statistical testing, and RS/ χ^2) for the primary analysis, with deep models as an optional secondary benchmark. Overall, prior work has studied embedding techniques, steganalysis models, generative steganography, and synthetic-image detection largely in isolation; our contribution is a controlled evaluation of how carrier origin influences detectability.

7. Relation to Curriculum

This project connects directly to **Computer Security** (AES-256-CBC and adversarial threat modeling), **AI and Machine Learning** (SRM+EC feature-based classification and cross-domain evaluation), **Software Engineering and Architectures** (modular, testable pipeline design), and **Statistics** (ANOVA, non-parametric testing, effect sizes, and confidence intervals for rigorous comparison).

8. Planning

Detailed Gantt charts are provided in Appendix A.

Phase 2: Implementation (Period 5, 30 Mar–15 May 2026, 7 weeks) covers three parallel workstreams: dataset construction and ML image generation (Wk 1–2), steganography pipeline implementation including LSB, DCT, and AES-256 encryption (Wk 2–3), and detection, analysis, and writing (Wk 3–7).

Phase 3: Completion (Project Period, 25 May–12 Jun 2026, 3 weeks) covers completing any remaining implementation, verifying and rerunning experiments, and finalising all deliverables (presentation slides, poster, and paper).

9. Minimal Passing Requirements

To satisfy the minimal passing criteria, the study must implement at least one encryption scheme, two embedding approaches (one spatial-domain and one frequency-domain), and domain-appropriate steganalysis detectors for the implemented embedding methods, and it must provide conclusive answers to RQ1 (carrier-source effect: real vs. ML) and RQ2 (generation-model effect within ML)

References

- [1] F. A. P. Petitcolas, R. J. Anderson, and M. G. Kuhn, "Information hiding: a survey," *Proc. IEEE*, vol. 87, no. 7, pp. 1062–1078, 1999.
- [2] A. Cheddad, J. Condell, K. Curran, and P. McKeivitt, "Digital image steganography: Survey and analysis of current methods," *Signal Process.*, vol. 90, no. 3, pp. 727–752, 2010.
- [3] M. Hussain, A. W. A. Wahab, Y. I. B. Idris, A. T. S. Ho, and K. H. Jung, "Image steganography in spatial domain: A survey," *Signal Process.: Image Commun.*, vol. 65, pp. 46–66, 2018.
- [4] J. Fridrich and J. Kodovský, "Rich models for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 3, pp. 868–882, 2012.
- [5] J. Kodovský, J. Fridrich, and V. Holub, "Ensemble classifiers for steganalysis of digital media," *IEEE Trans. Inf. Forensics Security*, vol. 7, no. 2, pp. 432–444, 2012.
- [6] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE CVPR*, pp. 10684–10695, 2022.
- [7] D. Podell et al., "SDXL: Improving latent diffusion models for high-resolution image synthesis," arXiv:2307.01952, 2023.
- [8] J. Chen et al., "PixArt- α : Fast training of diffusion transformer for photorealistic text-to-image synthesis," arXiv:2310.00426, 2023.
- [9] T. Karras, M. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, "Alias-free generative adversarial networks," in *Proc. NeurIPS*, vol. 34, pp. 852–863, 2021.
- [10] A. De, W. Kinzel, and I. Kanter, "Steganographic secret sharing via AI-generated photorealistic images," *EURASIP J. Wireless Commun. Netw.*, art. 108, 2022.
- [11] J. Fridrich, M. Goljan, and R. Du, "Detecting LSB steganography in color and grayscale images," *IEEE Multimedia*, vol. 8, no. 4, pp. 22–28, 2001.
- [12] A. Westfeld and A. Pfitzmann, "Attacks on steganographic systems," in *Proc. 3rd Int. Workshop Information Hiding*, LNCS 1768, pp. 61–76, 1999.
- [13] H. Wang and P. Moulin, "Steganalysis of block-DCT image steganography," in *Proc. IEEE SSP*, pp. 477–488, 2003.
- [14] N. Provos and P. Honeyman, "Hide and seek: An introduction to steganography," *IEEE Security Privacy*, vol. 1, no. 3, pp. 32–44, 2003.
- [15] B. Chen and G. W. Wornell, "Quantization index modulation: A class of provably good methods for digital watermarking and information embedding," *IEEE Trans. Inf. Theory*, vol. 47, no. 4, pp. 1423–1443, 2001.
- [16] S. Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, "CNN-generated images are surprisingly easy to spot... for now," in *Proc. IEEE CVPR*, pp. 8695–8704, 2020.
- [17] R. Corvi, D. Cozzolino, G. Zingarini, G. Poggi, K. Nagano, and L. Verdoliva, "On the detection of synthetic images generated by diffusion models," in *Proc. IEEE ICASSP*, pp. 1–5, 2023.
- [18] D.-T. Dang-Nguyen, C. Pasquini, V. Conotter, and G. Boato, "RAISE: A raw images dataset for digital image forensics," in *Proc. ACM MMSys*, pp. 219–224, 2015.
- [19] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. ECCV*, LNCS 8693, pp. 740–755, 2014.
- [20] P. Young, A. Lai, M. Hodosh, and J. Hockenmaier, "From image descriptions to visual denotations," *Trans. Assoc. Comput. Linguist.*, vol. 2, pp. 67–78, 2014.
- [21] Y. Luo et al., "Deep learning for steganalysis of diverse data types: A review," *Neurocomputing*, Elsevier, 2024.
- [22] M. Boroumand, M. Chen, and J. Fridrich, "Deep residual network for steganalysis of digital images," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 5, pp. 1181–1193, 2019.
- [23] V. Holub, J. Fridrich, and T. Denemark, "Universal distortion function for steganography in an arbitrary domain," *EURASIP J. Inf. Security*, art. 1, 2014.
- [24] P. Hu et al., "A coverless image steganography based on generative adversarial networks," *Electronics*, vol. 12, no. 5, art. 1253, 2023.
- [25] X. Liu et al., "Message-driven generative steganography using GAN," *IEEE Trans. Dependable Secure Comput.*, 2024.
- [26] X. Duan, D. Jia, B. Li, D. Guo, E. Zhang, and C. Qin, "Coverless steganography based on generative adversarial network," *EURASIP J. Image Video Process.*, art. 46, 2020.

A. Project Gantt Charts

Phase 2: Implementation

Period 5: 30 March – 15 May 2026 (7 weeks)

WBS	Task	Start	End	Days	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7
1	Data Construction										
1.1	Dataset collection (COCO/Flickr30k with captions)	30.03	10.04	10							
1.2	ML image generation (SDXL + PixArt- α)	30.03	10.04	10							
1.3	BRISQUE quality gate & normalisation	07.04	17.04	8							
2	Steganography Implementation										
2.1	LSB embedding pipeline ($k=1, 2$)	07.04	17.04	8							
2.2	DCT-QIM embedding pipeline	07.04	17.04	8							
2.3	AES-256-CBC payload encryption	14.04	17.04	4							
3	Detection & Analysis										
3.1	RS Analysis + χ^2 detector	14.04	24.04	8							
3.2	SRM + EC and DCT statistical detector setup	14.04	01.05	14							
3.3	Image quality metrics (PSNR/SSIM/FSIM)	21.04	01.05	8							
3.4	Encryption-effect experiments (RQ5)	28.04	08.05	8							
3.5	Statistical analysis (ANOVA, Wilcoxon)	28.04	08.05	8							
4	Writing & Revision										
4.1	Visualisations + report writing	04.05	15.05	8							
4.2	Buffer / final revision	11.05	15.05	5							

■ Data ■ Steganography ■ Detection ■ Evaluation ■ Analysis ■ Writing

Milestones: **M1** (end Wk 2): Dataset ready **M2** (end Wk 3): Embedding pipelines verified **M3** (end Wk 5): All experiments complete **M4** (end Wk 7): Report submitted

Phase 3: Completion

Project Period: 25 May – 12 June 2026 (3 weeks)

WBS	Task	Start	End	Days	Wk 1	Wk 2	Wk 3
1.1	Complete remaining implementation	25.05	05.06	10			
1.2	Verify results & rerun experiments	25.05	05.06	10			
1.3	Statistical verification & effect sizes	01.06	05.06	5			
2.1	Presentation slides	01.06	08.06	6			
2.2	Poster	05.06	10.06	4			
2.3	Final paper write-up	01.06	12.06	10			
2.4	Buffer / revision / submission	10.06	12.06	3			

■ Implementation ■ Verification ■ Analysis ■ Slides ■ Poster ■ Paper

Milestones: **M5** (end Wk 1): Implementation finalised **M6** (end Wk 2): Results verified, slides ready **M7** (end Wk 3): All deliverables submitted